

AQA A Level Computer Science NEA

Shayan Nezami

January 3, 2026

Contents

1	Analysis	3
1.1	Introduction	3
1.2	Client Overview	3
1.3	Client Interview	3
1.4	Specification	5
1.5	Technical Points	6
1.6	Program Structure Diagram	7
1.7	Transmission Procedure	7
1.7.1	Key Exchange	8
1.7.2	Symmetric Encryption	9
1.7.3	Verifying the Integrity of Communications	10
1.8	Compression	10
1.9	Detecting Unchanged Files	11
1.10	Encrypting Files	11
1.11	Class Diagram	11
1.12	Order of Completion	11
2	Documented Design and Technical Solution	13
2.1	Huffman Compression	13
2.1.1	Encoding and Decoding	13

2.1.2	Reading and Writing Bytes	13
2.1.3	UML Class Diagram	19
2.1.4	The solution	19

List of Figures

1	Program Structure Diagram	7
2	A flowchart depicting my Huffman encoding procedure	14
3	A flowchart depicting my Huffman decoding procedure	15
4	A UML class diagram for the compression program	19

Listings

1	Pseudocode for the ByteMaker class	16
2	Pseudocode for the ByteReader class	17
3	Selector program for compression	19
4	Code for the HuffmanNode class	20
5	Code for the ByteMaker class	21
6	Code for the ByteReader class	23
7	Code for the HuffmanEncoder class	24
8	Code for the HuffmanDecoder class	26

1 Analysis

1.1 Introduction

A backup utility is an essential software program for any computer system, allowing users to keep their files and data safe in the case of hardware failure, or various possible software issues. It is particularly important that these files are stored externally, on another device, in case of such hardware failure. I am interested in Linux systems, and so I will create a backup utility for storing files on an external server from a client PC. This will require various components, such as encrypting the files stored on the server, encrypting the transmission of files between client and server, and compressing the files being transferred.

The main benefits of the program I will create over existing systems are that my utility will be fully open source, so the function of the program can be independently inspected by any user, and that it will be tailored to my client's exact needs: many existing applications are either closed source, or too bloated with unnecessary features and unintuitive interfaces.

I will research this through two main methods: discussion with my client, and use of reliable sources. The latter may come from reputable books, websites, papers, or journals, and will be cited.

1.2 Client Overview

My client will be my mother – Elham Asadi – who informed me she is looking for such software. Therefore, I will tailor this program for her exact needs, which I will establish her specific requirements through a client interview. This has the benefit over existing systems of being exactly the format she would like, without additional features which add complexity and bloat to the final program, and usable with the exact desired interface.

1.3 Client Interview

Below is a record of the transcript of my first interview with my client. I will use this to inform the design requirements for this backup utility.

Q: I understand that you are looking for a backup utility?

A: Yes, I'd like to be able to create external backups of my system on a separate device.

Q: What devices would you like to create backups of?

A: My laptop and desktop PC.

Q: What operating system are those devices running?

A: Arch Linux, and Ubuntu 22.04 respectively, but I would like this to work on any modern Linux distribution - I may want to change in the future!

Q: Where would you like the backups to be stored?

A: I have a fairly old laptop which I use to run a few home server utilities. I'd like to also store the backups here, if possible.

Q: Can you be more specific about the laptop?

A: It's around 15-20 years old, I'm not exactly sure, with 4GB of RAM. Also, it's running Ubuntu server.

Q: Would you like a third-party server in a data centre to store the backups instead?

A: Absolutely not. The key here is that this is all fully local to me, and that I am the only one with access to the backups. Many current backup utilities lack this!

Q: Would you want the files stored to be encrypted in transmission, and while stored?

A: Certainly, this is crucial! Nobody but me should be able to decipher the files stored.

Q: Do you have any other priorities?

A: File transfers should be as quick as possible, and as little space as possible should be used on the server device. Oh, and I want to be sure that transfers have been successful.

Q: What type of files will you be backing up?

A: Well, I will be using this for my whole system, so all sorts of files. I guess there will be text files, like system configuration files, image files, video files, and raw binary executables.

Q: Okay. The non-text files you were talking about, are they already compressed?

A: Yes actually, almost all of my audio and video files are already compressed.

Q: Very good, thanks. What about the user interface, what form should this take?

A: I want a very simple command line interface, with no messy graphical elements. It could have an interactive mode, but there should certainly be a way to use it non-interactively, with just command line flags, so that I can use it in scripts.

Q: Finally, is it important to you for this project to be finished by a certain date?

A: Absolutely. It is essential for this to be finished by [date I actually finish]. This is one of my key priorities, since that is when I will purchase a laptop for a friend, and I'd like to have the backup software ready for them when they receive it.

Q: Does this mean I need to ensure the program works for a different laptop's specifications?

A: No, I don't think so, that laptop should be very similar to my one, that I've already described.

1.4 Specification

Based on the requirements established in the client interview, I have established the following specification points. These will guide me throughout the development process, and can serve as a metric against which the final product is evaluated.

1. The program should allow the user to create file backups on a server, run by the client themselves,
 - (a) Both the client and the server programs should be created,
 - (b) They should be able to work together, with a server being able to store the user's backed up files,
2. The client program should run on modern Linux systems, usable on a laptop or desktop computer,
 - (a) An installable binary package should be created that is executable on amd64 Linux systems,
 - (b) The client should not at any point use more than [some]MB of RAM, or more than [some]% of CPU time while running on a modern laptop (i9-12950HX processor for a test case),
3. It should be usable with an intuitive command-line interface, not requiring interactive input,
 - (a) All required functions of the program should be able to be passed in through command line flags and parameters,
 - (b) There can be an optional interactive mode, but this should not allow any more functionality than the non-interactive mode,
 - (c) There should be no graphical interface,
4. The server should be able to run on a low-end laptop 10-15 years old,
 - (a) The program should avoid reading and writing to the system drives as much as possible, since most such devices use HDDs,
 - (b) The server program should not at any point use more than [some]MB of RAM, or more than [some]% of CPU time while running on such a device (i5-[something] processor as a test case),
5. All file transfers should be secure, so a man-in-the-middle would not be able to intercept data,

- (a) Any data intercepted should be of a format that the interceptor would not be able to decipher, or gain any useful information from,
 - (b) The methods used should be verified to be correct,
- 6. The files stored on the server should all be encrypted with a key only accessible to the client,
 - (a) Only the client should be able to decipher the files stored on the server,
 - (b) The encryption should not increase the size of the files significantly - this will be defined as the larger of 1MB, and 2% of the original file size,
- 7. Files stored on the server should be compressed such that they use a reasonable amount of storage,
 - (a) They should be at most as large as the uncompressed files, i.e. file size should never increase following compression,
 - (b) The client should be able to uncompress any file when needed by the user, storing any needed details about the compression within the file,
- 8. File transfers should be verifiably correct, so data is not corrupted during transmission,
 - (a) Once a file has been transferred to the server, it should be compared with the original file on the client device,
 - (b) This checking should be with a much smaller version of the file than that which was originally sent for most files – it should be up to [some]kB,
- 9. File transfers should be as quick as possible, with duplicate data not sent,
 - (a) Files that remain unchanged on the server since the last backup should not be sent again, rather a message should confirm that the file should remain as it was in the previous backup.

1.5 Technical Points

Based on the specification points established, I have extracted the key technical challenges that will be involved in building the software below.

1. Implement RSA style transmission with asymmetric encryption for key exchange between client and server,
2. Implement symmetric encryption between client and server after keys have been established,
3. Use hashing to verify the integrity of messages transferred,
4. Ensure unchanged files are not transferred,
5. The files stored on the server should all be encrypted with a key only accessible to the client,
6. Implement a lossless compression algorithm for files stored in the server.

1.6 Program Structure Diagram

This diagram (Figure 1) shows how these main technical blocks will interact with each other in the client and server programs. While they will run different programs, these share a common transmission and receiving procedure.

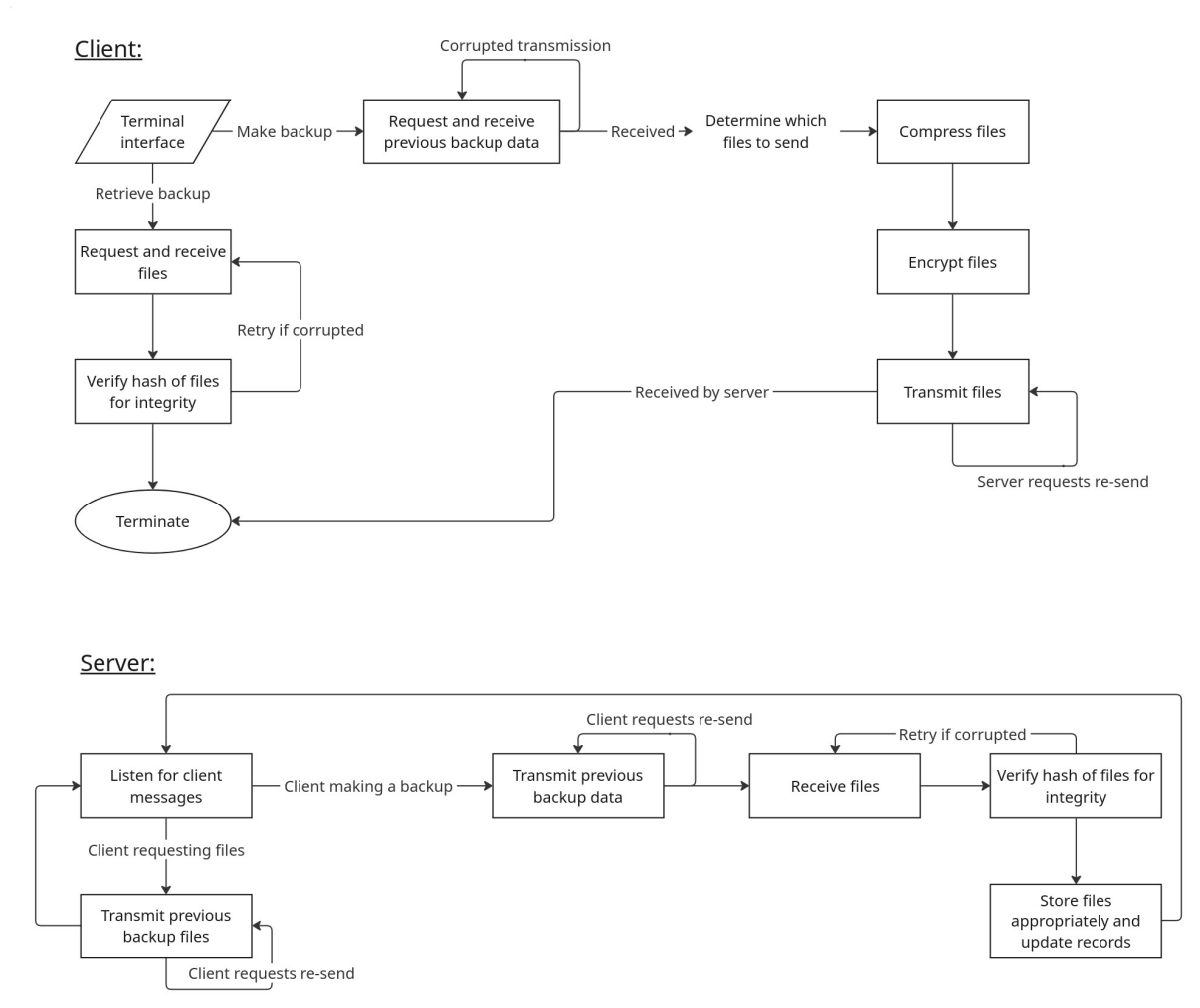


Figure 1: Program Structure Diagram

I will now consider how I can implement these main blocks.

1.7 Transmission Procedure

This is one of the largest components of the program, which is used for any and all communications between the client and server devices. There are some key points that need to be individually considered:

1. There needs to be a procedure used to exchange keys between the sender and receiver, perhaps in a similar style to the RSA algorithm, providing asymmetric encryption,
2. There must be a symmetric encryption algorithm used to encrypt communications following the establishment of keys,

3. The communications should be verified to be correct, using a method to check that there has not been any corruption during data transfer.

1.7.1 Key Exchange

The three algorithms I will consider for this are RSA, Diffie-Hellman Key Exchange, and Elliptic Curve Cryptography.

These algorithms are commonly used, and very well documented. All three rely on mathematical problems that are intractable to solve without some prior knowledge: RSA relies on the difficulty in factoring a large semiprime integer (one with exactly two prime factors), while Diffie-Hellman and elliptic curve systems rely on the discrete logarithm problem.

While elliptic curve cryptography can provide equivalent security with much smaller keys, and requires less processing power for the sender and receiver, this is unlikely to be at all impactful on the small scale of this backup utility, where this asymmetric encryption is just used to share a symmetric encryption key. However, these algorithms are tough to implement, and involve very high-level mathematics. This could increase the time taken to deliver the program past the client's deadline, and adds more avenues for potential bugs to develop. This can be very dangerous, as shown in the following paper [1], where a small error in the implementation led to researchers being able to extract the full key.

I will now consider the basic implementation details of the RSA and Diffie-Hellman algorithms to aid in my decision-making.

RSA

1. The receiver generates two large primes, p, q , then computes their product, $n = pq$.
2. Euler's Totient function, $\phi(p, q) = (p - 1)(q - 1)$, is applied to n .
3. An encryption exponent, e , is selected such that $1 < e < \phi(n)$, and e is relatively prime with $\phi(n)$. Note that $a, b \in \mathbb{Z}^+$ are relatively prime $\iff \nexists x \in \mathbb{Z} : x > 1, x | a, x | b$.
4. A decryption exponent, d , is calculated such that $de = 1 \pmod{\phi(n)}$. The extended Euclidian algorithm provides a method for this.
5. The public key is formed from (n, e) and the private key (n, d) . The receiver sends their public key only to the sender.
6. The encrypted ciphertext, C , is calculated through $C = M^e \pmod{n}$, where M is some numerical representation of the message.
7. The ciphertext is decrypted into the plaintext message, M , by the calculation $M = C^d \pmod{n}$. [2]

Now, the sender encrypts their message with the receiver's public key, and then only the receiver can decrypt it since only they have the private key. Computing $\phi(n)$ is infeasible without

knowing the primes, p, q , that n is a product of, and this is also infeasible to compute due to the large size of the primes. This means no other party can calculate the private key from the public key.

Diffie-Hellman key exchange

1. Either the sender or the receiver randomly generates a large prime, p , and a large number, g , and sends these to the other.
2. The sender and receiver then generate random secrets s_a and s_b respectively, for $s_a, s_b \in \mathbb{Z}^+$.
3. The sender sends $a = g^{s_a} \pmod{p}$ to the receiver, who computes $a^{s_b} = g^{s_a s_b} \pmod{p}$.
4. The receiver sends $b = g^{s_b} \pmod{p}$ to the sender, who computes $b^{s_a} = g^{s_a s_b} \pmod{p}$.
5. Now, they both have the shared secret, $S = g^{s_a s_b} \pmod{p}$, which can be used to establish a shared key. [3]

An attacker would have access to g , p , $g^{s_a} \pmod{p}$, $g^{s_b} \pmod{p}$, and need to compute $S = g^{s_a s_b} \pmod{p}$ to compromise the conversation. A quick solution to the Discrete Logarithm Problem would imply a quick solution to this problem, but one is not currently known. This is central to the security of this key exchange, similarly to the prime factorisation problem for RSA.

As can be seen, RSA and Diffie-Hellman are very similar in security, and difficulty of implementation, however the advantage of RSA is that it can also be used to provide signed certificates to prove the authenticity of the sender. This has not been an aim as of yet, but using RSA will make it easier to implement this if desired in the long-term, or at the end of the project if there is sufficient time.

1.7.2 Symmetric Encryption

Following the establishment of keys, it is greatly advantageous to use a symmetric encryption system rather than continuing with asymmetric systems, since these are much faster and more efficient. A similar process is used by the TLS protocol, which secures more internet communications.

There are a great number of available symmetric encryption algorithms, including the Caesar Cipher, AES, ChaCha20, and Blowfish, among others.

The Caesar cipher can be immediately eliminated, since it is trivial to break with even very simple cryptanalysis. However, the other algorithms are very similar in that they all provide secure encryption that is very tough to break. The final choice of algorithm should be one that is well-documented, simple to implement, and efficient to run.

However, AES has been used as the basis of internet security for decades, and as such there has been great research into verifying its security, and ensuring implementations do not contain vulnerabilities. Therefore, I will use this for my symmetric encryption needs.

1.7.3 Verifying the Integrity of Communications

It is necessary to ensure all communications have arrived at the desired recipient uncorrupted, exactly as they were sent. Therefore, some method must be used to ensure that this is the case.

Some simple methods for this include majority voting, where each bit is sent an odd number (> 1) times, and the most common bit is accepted, and parity bits, where a bit is added to the end of each string of defined length to ensure that the number of 0s or 1s is either even or odd as desired.

However, these methods allow multiple mistakes to “cancel” each other out, and say little about the correctness of the whole message. A more robust method is to compute some form of hash from the message, and send that directly after it. The receiver will also compute a hash of the data received, and compare it to the hash sent. If these match, then the data has arrived intact. This verifies that the whole message has arrived safely, while requiring comparatively little additional data to be transferred. Plus, since there is no need for a cryptographic hash function (it is not important if the hash can be reversed), these algorithms can run very quickly.

[describe the function of the algorithm]

1.8 Compression

Of course, I will want to use a lossless algorithm, since a backup program should not change the files being backed up – the whole point is to provide a safe copy of the actual system files themselves. Plus, lossy compression of important system files would be catastrophic.

There is also the key problem that many files being backed up will already be compressed. The system must be able to deal with these, not producing output with larger file size than the uncompressed versions.

There is also the important consideration of whether each file is individually compressed, or whether the whole block of files is compressed together.

There are two broad options I can take based on the above. I could either compress the whole block of data, or each file individually, with some metadata at the beginning of the file outlining what procedure (if any) was used.

The first option will not be very useful, since different file types can be effectively compressed using vastly different algorithms, so I will not be able to achieve significant size reductions in this way. Therefore, I believe the right choice is to compress files individually.

This means that I would need to write different algorithms for different file types, and have a way of detecting files that have already been compressed. However, the discussion with my client revealed that she already compresses the majority of non-text files on her system. So, I have decided to just implement compression for plaintext files, where the most space savings can be made, while minimising the unnecessary processing taking place, so the program executes as quickly as possible. This will also reduce development time, allowing me to finish the program in time for my client’s strict deadline.

Three lossless plaintext compression algorithms I will consider as the basis of my algorithm are Run-Length Encoding, Huffman Coding, and Prediction by Partial Matching.

Some depth on the algorithms, how they work, and choosing Huffman

1.9 Detecting Unchanged Files

I had initially come up with two methods that could be used for this. These were comparing the files (the previously backed-up version and the new file) line by line, and comparing a hash of the files.

The first method is pointless, since the file could just be overwritten in the same time. Therefore, I was planning on using the second method. However, browsing forums for a quicker way revealed that I can simply compare the timestamps of the file to the time of the previous backup. This is much quicker, and does not require any communication with the backup server beyond getting the date of the last backup and list of files in that backup, which could be locally stored on the client anyway. [\[4\]](#)

As such, this is the method I will use.

1.10 Encrypting Files

I can use the same algorithm I have implemented for providing symmetric encryption for transmission after key exchange for this purpose, to minimise redundant work. The only difference is that the encryption key will be stored somewhere on the client device, rather than only being used in the one-time communication.

1.11 Class Diagram

Draw one

1.12 Order of Completion

This is a rough timeline of how long I expect it would take me to complete the sections outlined above.

1. Compression - 2 weeks
2. Basic symmetric encryption - 1 week
3. RSA style key exchange - 2 weeks
4. Hashing to verify transmission - 1 week
5. Terminal interface - 1 week

6. Putting it all together - 2 weeks

Edit to remove(?) symmetric encryption

2 Documented Design and Technical Solution

2.1 Huffman Compression

As decided upon in my Analysis (Section 1.8), I will use Huffman Coding to compress all plaintext files that are being backed up. The compression and decompression will occur on the client-side, before the files are sent, and after they are received.

The benefits of this are twofold. Firstly, this means less data is transmitted, reducing the risk of corruption, and speeding up the backup process. Secondly, as identified in my Client Interview (Section 1.3), my client will run the server on hardware that is significantly less powerful than their laptop, which acts as the client. Therefore, it is advantageous to run as much processing as possible on the client-side.

2.1.1 Encoding and Decoding

This website [5] provides a thorough written explanation of the Huffman algorithm. Using this to better understand the process, I have created a flowchart depicting how I can implement the encoding (Figure 2), and the decoding (Figure 3).

These flowcharts include implementation details, such as how dictionaries, stacks, priority queues, and recursive subroutines are used to carry out some of the high-level tasks described in the source. My implementation also ensures the compression does not (significantly) increase the size of the files, by using a 1-bit flag at the start of the file to indicate whether compression has been used, and not using any compression if it would increase the file size overall. Do note that the final file will still be one byte larger in this case due to the extra bit added, but this is insignificant and unavoidable.

2.1.2 Reading and Writing Bytes

These implementations assume writing bits to, and reading bits from, files is a trivial task. However, this proved to be quite challenging, since one can only write to, and read from, files one byte at a time in C#, so I needed to implement additional helper methods for this.

Therefore, I constructed pseudocode for ByteMaker (Listing 1) and ByteReader (Listing 2) classes, which respectively allow writing to and reading from binary files.

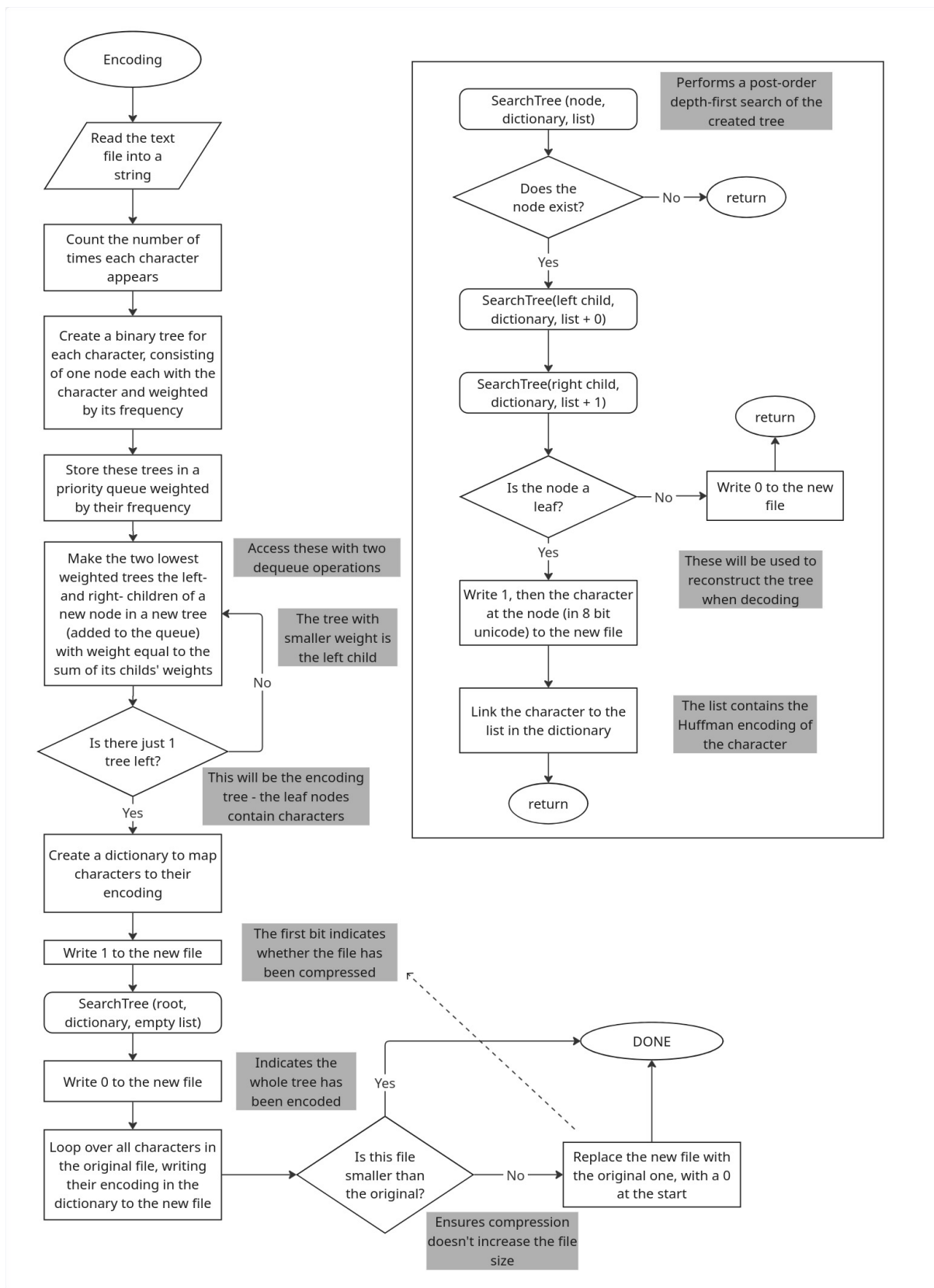


Figure 2: A flowchart depicting my Huffman encoding procedure

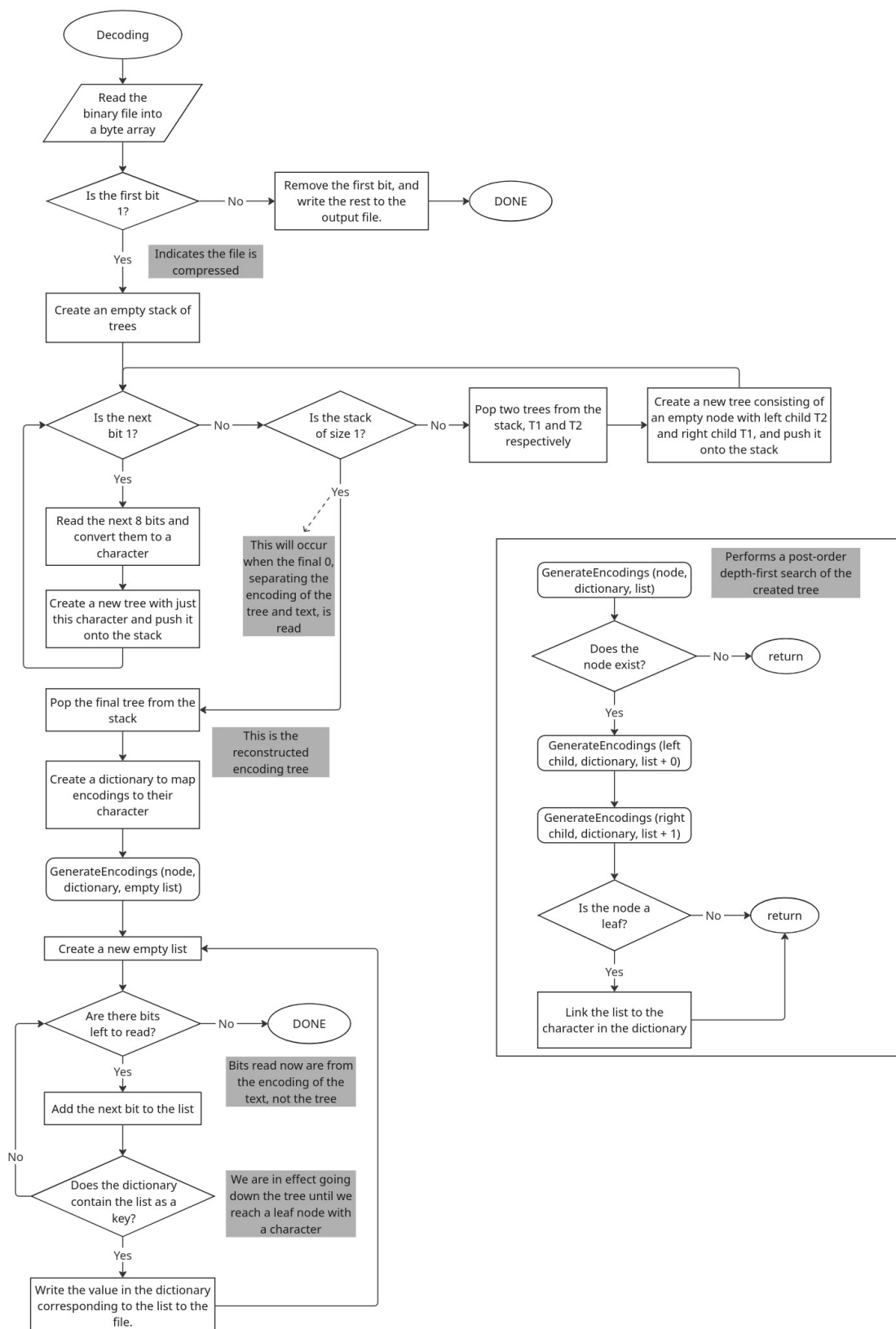


Figure 3: A flowchart depicting my Huffman decoding procedure

```

1 CLASS ByteMaker
2     PRIVATE INTEGER ARRAY _bits
3     PRIVATE INTEGER _bitIndex
4     PRIVATE INTEGER _trailingZeros
5
6     PRIVATE BYTE LIST _bytes
7
8     PUBLIC CONSTRUCTOR ByteMaker (BOOLEAN isCompressed)
9         _bitIndex <- 4
10        IF (isCompressed = TRUE) THEN
11            _bits <- [1, 0, 0, 0, 0, 0, 0, 0]
12        ELSE
13            _bits <- [0, 0, 0, 0, 0, 0, 0, 0]
14        ENDIF
15    ENDCONSTRUCTOR
16
17    PUBLIC SUBROUTINE Add (INTEGER bit)
18        IF (bit <> 1 AND bit <> 0) THEN
19            RETURN
20        ENDIF
21
22        _bits[_bitIndex] <- bit
23        _bitIndex <- _bitIndex + 1
24
25        IF (_bitIndex = 8) THEN
26            updateByte()
27        ENDIF
28    ENDSUBROUTINE
29
30    PUBLIC SUBROUTINE Add (INTEGER LIST bits)
31        FOR (INTEGER bit IN bits)
32            Add(bit)
33        ENDFOR
34    ENDSUBROUTINE
35
36    PUBLIC SUBROUTINE Add (CHARACTER char)
37        BYTE charByte <- char TO BYTE
38        FOR (INTEGER i <- 7 TO 0 STEP -1)
39            INTEGER bit <- (RIGHTSHIFT charByte i) AND 1
40
41            Add(bit)
42        ENDFOR
43    ENDSUBROUTINE
44
45    PUBLIC SUBROUTINE Export()
46        updateByte()
47
48        BYTE trailingZerosByte <- _trailingZeros TO BYTE
49        _bytes[0] <- _bytes[0] OR (LEFTSHIFT trailingZerosByte
50            4)

```



```

50
51         RETURN (_bytes TO ARRAY)
52     ENDSUBROUTINE
53
54     PRIVATE SUBROUTINE updateByte()
55         BYTE newByte
56         FOR (INTEGER i <- 0 TO 7 STEP 1)
57             BYTE mergeByte <- (LEFTSHIFT _bits[i] 7-i) TO
58                 BYTE
59             newByte <- newByte OR mergeByte
60         ENDFOR
61         ADD newByte TO _bytes
62
63         _bits <- [0, 0, 0, 0, 0, 0, 0, 0]
64         _trailingZeros <- 8 - _bitIndex
65         _bitIndex <- 0
66     ENDSUBROUTINE
67 ENDCLASS

```

Listing 1: Pseudocode for the ByteMaker class

The ByteMaker provides methods to write single bits (as integers 0/1), lists of bits, and characters to a private list of bytes, which it can "export" upon completion, returning a byte array that can be written to the binary file. The class holds an 8 long array of integers to represent the current byte, and this is added to the byte array when it is filled (when the index used to write to it reaches 8). This is done by converting it into a byte using bitwise operations (shifting each bit to its position, then performing an OR operation with the byte being constructed).

When all desired bits have been written to the file, it is likely that this is not at the end of a byte, and so the remaining bits are left as 0, since you can only write to files in bytes. However, the number of such bits need to be stored in some form, so it is clear which bits to ignore when decoding. Therefore, bits 1-3 of the first byte are kept clear for writing a 3-bit integer between 0-7 (the possible number of trailing bits) at the end. Note that bit 0 is used to indicate whether the file is being compressed in the first place. This is why the index begins at 4 rather than 0 for the first bit.

```

1 CLASS ByteReader
2     PRIVATE BYTE LIST _bytes
3     PRIVATE INTEGER _bitIndex
4     PRIVATE INTEGER _trailingNum
5
6     PUBLIC CONSTRUCTOR ByteReader (BYTE ARRAY bytes)
7         _bytes <- bytes TO LIST
8         _bitIndex <- 0
9
10        INTEGER firstBit <- Read()
11        INTEGER a <- Read()
12        INTEGER b <- Read()
13        INTEGER c <- Read()
14        _trailingNum <- 4*a + 2*b + c

```

```

15         IF (firstBit = 0) THEN
16             RETURN FALSE
17         ELSE
18             RETURN TRUE
19         ENDIF
20     ENDCONSTRUCTOR
21
22
23     PUBLIC SUBROUTINE Read()
24         IF (LENGTH _bytes = 0) THEN
25             RETURN -1
26         ENDIF
27         IF (LENGTH _bytes = 1 AND _bitIndex >= 8 -
28             _trailingNum) THEN
29             RETURN -1
30         ENDIF
31
32         INTEGER bit <- (RIGHTSHIFT _bytes[0] (7 - _bitIndex))
33             AND 1
34         _bitIndex <- _bitIndex + 1
35
36         RETURN bit
37     ENDSUBROUTINE
38
39     PUBLIC SUBROUTINE ReadChar()
40         BYTE nextByte <- 0
41
42         FOR (INTEGER i <- 0 TO 7 STEP 1)
43             INTEGER bit <- Read()
44             IF (bit = -1) THEN
45                 RETURN ''
46             ELSE IF (bit = 1) THEN
47                 nextByte <- nextByte OR (LEFTSHIFT 1
48                     (7-i))
49             ENDIF
50         ENDFOR
51
52         RETURN (nextByte TO CHARACTER)
53     ENDSUBROUTINE
54
55     PRIVATE SUBROUTINE Update()
56         IF (_bitIndex <> 8) THEN
57             RETURN
58         ENDIF
59
60         REMOVE INDEX 0 FROM _bytes
61         _bitIndex <- 0
62     ENDSUBROUTINE
63
64 ENDCLASS

```

Listing 2: Pseudocode for the ByteReader class

This `ByteReader` class acts as the opposite of the `ByteMaker`: it takes in a byte array when being instantiated, and provides methods for reading the next bit and character (8 bits). The constructor returns a boolean value indicating whether the first bit is 0 or 1, and so whether compression was used on the file. It also uses the next 3 bits to determine the number of trailing 0s at the end of the file, so they can be ignored when reading. This is indicated by the `Read` method returning -1 once those characters have been reached.

The `Read` method uses bitwise operations to get the bit at a certain index from the byte at index 0 of the list. Index 0 is always used as the `Update` method removes bytes that have already been fully read from the start of the list. The right shift operation places the desired bit at the end of the byte, and performing an AND with 1 returns the value of the last bit of the right-shifted byte, as desired.

2.1.3 UML Class Diagram

Figure 4 shows a UML class diagram depicting my design for this component of the program. No inheritance, composition, or aggregation is used, but the `HuffmanNode` constructor and the `ByteMaker` `Add` method show polymorphism by having overloads.

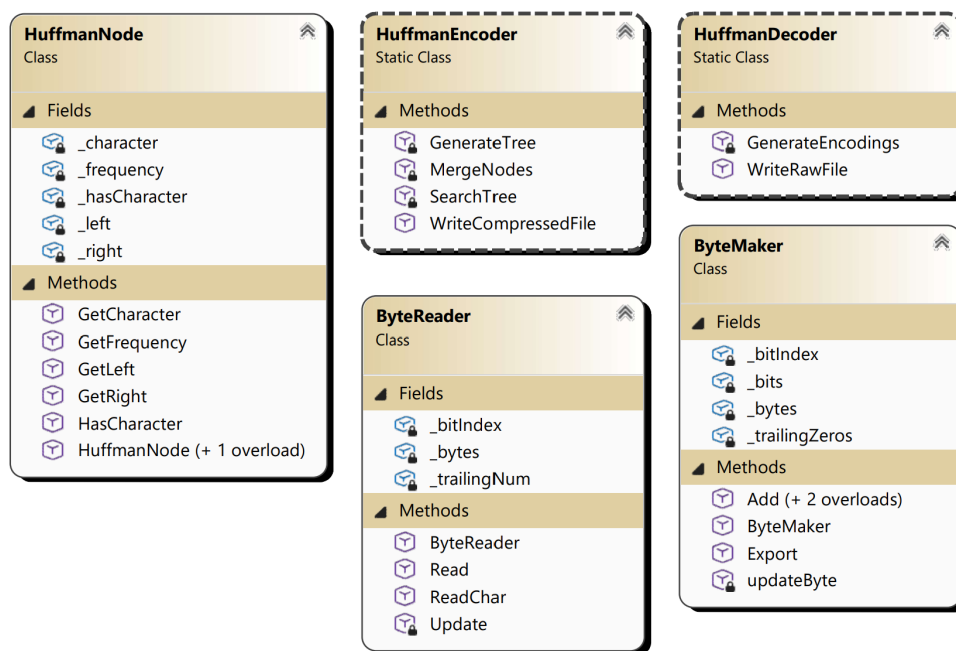


Figure 4: A UML class diagram for the compression program

2.1.4 The solution

```

1 bool Program()
2 {
3     if (args.Length < 3) return false;
4
5     switch (args[0] ?? "")

```

```

6      {
7          case "-e":
8              string fileText;
9
10             try
11             {
12                 fileText = File.ReadAllText(args[1]);
13             }
14             catch
15             {
16                 Console.WriteLine("Could not read from " + args[1]);
17                 return true;
18             }
19
20             HuffmanEncoder.WriteCompressedFile(fileText, args[2]);
21             break;
22
23             case "-d":
24                 byte[] fileBytes;
25
26                 try
27                 {
28                     fileBytes = File.ReadAllBytes(args[1]);
29                 }
30                 catch
31                 {
32                     Console.WriteLine("Could not read from " + args[1]);
33                     return true;
34                 }
35
36                 HuffmanDecoder.WriteRawFile(fileBytes, args[2]);
37                 break;
38
39             default:
40                 return false;
41         }
42
43         return true;
44     }
45
46     if (!Program())
47         Console.WriteLine("Error! Format:\n-[e,d] [input file] [output\npath]");

```

Listing 3: Selector program for compression

```

1 class HuffmanNode
2 {
3     private char _character;
4     private int _frequency;
5     private HuffmanNode _left;

```

```

6     private HuffmanNode _right;
7
8     private bool _hasCharacter; // "isLeaf"
9
10    public HuffmanNode(char character, int frequency = 1)
11    {
12        _character = character;
13        _frequency = frequency;
14
15        _hasCharacter = true;
16    }
17
18    public HuffmanNode(HuffmanNode left, HuffmanNode right, int
19        frequency = 1)
20    {
21        _left = left;
22        _right = right;
23        _frequency = frequency;
24
25        _hasCharacter = false;
26    }
27
28    public int GetFrequency() => _frequency;
29
30    public HuffmanNode GetLeft() => _left;
31    public HuffmanNode GetRight() => _right;
32
33    public bool HasCharacter() => _hasCharacter;
34    public char GetCharacter() => _character;
35
36    }
37
38    // NOTE: tree is just a "top-node" - links to left/right

```

Listing 4: Code for the HuffmanNode class

```

1  class ByteMaker
2  {
3      private int[] _bits;
4      private int _bitIndex;
5      private int _trailingZeros;
6
7      private List<byte> _bytes;
8
9      public ByteMaker(bool isCompressed)
10     {
11         _bits = [isCompressed ? 1 : 0, 0, 0, 0, 0, 0, 0, 0]; // length
12             8
13         // first bit indicates compressed, next 3 bits for trailing
14             zeros
15         _bitIndex = 4;
16         _trailingZeros = 0;

```

```

15
16     _bytes = new List<byte>();
17 }
18
19 public void Add(int bit)
20 {
21     if (bit != 1 && bit != 0) return;
22
23     _bits[_bitIndex++] = bit;
24
25     if (_bitIndex == 8) updateByte();
26 }
27
28 public void Add(List<int> bits)
29 {
30     foreach (int bit in bits) Add(bit);
31 }
32
33 public void Add(char character)
34 {
35     byte characterByte = (byte)character;
36
37     for (int i = 7; i >= 0; i--)
38     {
39         // right-shift, then AND with 1 to get "last" bit
40         int bit = (characterByte >> i) & 1;
41
42         Add(bit);
43     }
44 }
45
46 public byte[] Export()
47 {
48     updateByte();
49
50     // push to bits 1-4, OR since already 0
51     _bytes[0] |= (byte)(_trailingZeros << 4);
52
53     return _bytes.ToArray();
54 }
55
56 private void updateByte()
57 {
58     byte newByte = new byte();
59
60     for (int i = 0; i < 8; i++)
61     {
62         // left-shift, then OR with existing byte
63         newByte |= (byte)(_bits[i] << (7-i));
64     }
65

```

```

66         _bytes.Add(newByte);
67
68         _bits = [0, 0, 0, 0, 0, 0, 0, 0]; // length 8
69         _trailingZeros = 8 - _bitIndex;
70         _bitIndex = 0;
71     }
72 }

```

Listing 5: Code for the ByteMaker class

```

1  class ByteReader
2  {
3      private List<byte> _bytes;
4      private int _bitIndex;
5      private int _trailingNum;
6
7      public ByteReader(byte[] bytes, out bool compressed)
8      {
9          _bytes = bytes.ToList();
10         _bitIndex = 0;
11
12         int firstBit = Read();
13
14         if (firstBit == 0) compressed = false;
15         else compressed = true;
16
17         int a = Read();
18         int b = Read();
19         int c = Read();
20
21         _trailingNum = a*4 + b*2 + c;
22     }
23
24     public int Read()
25     {
26         if (_bytes.Count() == 0) return -1;
27
28         // don't read padding zeros
29         if (_bytes.Count() == 1 && _bitIndex >= 8 - _trailingNum)
30             return -1;
31
32         int bit = (_bytes[0] >> (7 - _bitIndex++)) & 1;
33
34         Update();
35
36         return bit;
37     }
38
39     public char? ReadChar()
40     {
41         byte nextByte = 0;

```

```

41
42     for (int i = 0; i < 8; i++)
43     {
44         int bit = Read();
45         if (bit == -1) return null;
46
47         // shift 1 to its position and OR with existing byte
48         if (bit == 1) nextByte |= (byte)(1 << (7-i));
49     }
50
51     return (char)nextByte;
52 }
53
54 private void Update()
55 {
56     if (_bitIndex != 8) return;
57
58     _bytes.RemoveAt(0);
59     _bitIndex = 0;
60 }
61 }

```

Listing 6: Code for the ByteReader class

```

1 static class HuffmanEncoder
2 {
3     public static void WriteCompressedFile(string text, string path)
4     {
5         HuffmanNode tree = GenerateTree(text);
6
7         // add encoding of tree
8         ByteMaker byteMaker = new ByteMaker(true);
9
10        Dictionary<char, List<int>> CharacterEncodings =
11            new Dictionary<char, List<int>>();
12
13        SearchTree(tree, byteMaker, CharacterEncodings, new
14            List<int>());
15        byteMaker.Add(0);
16
17        // add encoding of text
18        foreach (char c in text) byteMaker.Add(CharacterEncodings[c]);
19
20        byte[] bytes = byteMaker.Export();
21
22        if (bytes.Length > text.Length)
23        {
24            ByteMaker textByteMaker = new ByteMaker(false);
25
26            foreach(char c in text) textByteMaker.Add(c);

```



```

27         bytes = textByteMaker.Export();
28     }
29
30     try
31     {
32         using (BinaryWriter binaryWriter =
33             new BinaryWriter(File.Open(path, FileMode.Create)))
34         {
35             binaryWriter.Write(bytes);
36         }
37     }
38     catch
39     {
40         Console.WriteLine("Error: Could not write to " + path);
41     }
42 }
43
44 private static HuffmanNode GenerateTree(string text)
45 {
46     // link characters to their count
47     Dictionary<char, int> CharacterCounts = new Dictionary<char,
48         int>();
49
50     foreach (char c in text)
51     {
52         if (CharacterCounts.ContainsKey(c)) CharacterCounts[c]++;
53         else CharacterCounts[c] = 1;
54     }
55
56     // place characters in priorityqueue (based on freq) to
57     // generate the tree
58     PriorityQueue<HuffmanNode, int> NodeQueue = new
59         PriorityQueue<HuffmanNode, int>(); // "base" trees of size 1
60
61     foreach (KeyValuePair<char, int> kvp in CharacterCounts)
62     {
63         NodeQueue.Enqueue(new HuffmanNode(kvp.Key, kvp.Value),
64             kvp.Value);
65     }
66
67     HuffmanNode tree = MergeNodes(NodeQueue); // tree = top node
68     // of tree
69
70     return tree;
71 }
72
73 // recursively merge nodes to make a tree
74 private static HuffmanNode MergeNodes(PriorityQueue<HuffmanNode,
75     int> trees)
76 {
77     if (trees.Count == 1) return trees.Dequeue();
78 }

```

```

72     HuffmanNode tree1 = trees.Dequeue();
73     HuffmanNode tree2 = trees.Dequeue();
74
75     int newFreq = tree1.GetFrequency() + tree2.GetFrequency();
76
77     HuffmanNode newTree = new HuffmanNode(tree1, tree2, newFreq);
78
79     trees.Enqueue(newTree, newFreq);
80
81     return MergeNodes(trees);
82 }
83
84 // fill byteMaker with encoded tree, and CharacterEncodings with
85 // bits encoding characters, uses post order DFS
86 private static void SearchTree(HuffmanNode tree, ByteMaker
87     byteMaker, Dictionary<char, List<int>> CharacterEncodings,
88     List<int> current)
89 {
90     if (tree == null) return;
91
92     SearchTree(tree.GetLeft(), byteMaker, CharacterEncodings,
93         current.Append(0).ToList());
94     SearchTree(tree.GetRight(), byteMaker, CharacterEncodings,
95         current.Append(1).ToList());
96
97     if (tree.HasCharacter()) // is leaf
98     {
99         byteMaker.Add(1);
100         byteMaker.Add(tree.GetCharacter());
101
102         CharacterEncodings[tree.GetCharacter()] = current;
103     }
104     else
105     {
106         byteMaker.Add(0);
107     }
108 }
109 }

```

Listing 7: Code for the HuffmanEncoder class

```

1 static class HuffmanDecoder
2 {
3     public static void WriteRawFile(byte[] bytes, string path)
4     {
5         bool fileCompressed;
6
7         ByteReader byteReader = new ByteReader(bytes, out
8             fileCompressed);
9         string rawFile = "";

```

```

9
10     if (!fileCompressed)
11     {
12         while (true)
13         {
14             char? nextChar = byteReader.ReadChar();
15
16             if (nextChar == null) break;
17
18             rawFile += nextChar;
19         }
20     }
21     else
22     {
23         Stack<HuffmanNode> NodeStack = new Stack<HuffmanNode>();
24
25         while (true)
26         {
27             int nextBit = byteReader.Read();
28
29             if (nextBit == 1)
30             {
31                 char? nextChar = byteReader.ReadChar();
32                 if (nextChar == null) break;
33
34                 NodeStack.Push(new HuffmanNode((char)nextChar));
35             }
36             else
37             {
38                 if (NodeStack.Count() == 1) break;
39
40                 HuffmanNode right = NodeStack.Pop();
41                 HuffmanNode left = NodeStack.Pop();
42
43                 NodeStack.Push(new HuffmanNode(left, right));
44             }
45         }
46
47         HuffmanNode tree = NodeStack.Pop(); // there should be
48         just one node left
49
50         Dictionary<string, char> EncodingCharacters = new
51             Dictionary<string, char>(); // maps encodings to
52             character
53         GenerateEncodings(tree, EncodingCharacters, "");
54
55         string currentEncoding = ""; // string, not int array, as
56         arrays/lists compared by instance, not contents
57
58         while (true)
59         {

```

```

56         int nextBit = byteReader.Read();
57         if (nextBit == -1) break;
58
59         currentEncoding += nextBit.ToString();
60
61         if (!EncodingCharacters.ContainsKey(currentEncoding))
62             continue;
63
64         rawFile += EncodingCharacters[currentEncoding];
65
66         currentEncoding = "";
67     }
68
69     try
70     {
71         File.WriteAllText(path, rawFile);
72     }
73     catch
74     {
75         Console.WriteLine("Error: Could not write to " + path);
76     }
77 }
78
79 private static void GenerateEncodings(HuffmanNode tree,
80     Dictionary<string, char> EncodingCharacters, string current)
81 {
82     if (tree == null) return;
83
84     GenerateEncodings(tree.GetLeft(), EncodingCharacters, current
85         + "0");
86     GenerateEncodings(tree.GetRight(), EncodingCharacters, current
87         + "1");
88
89     if (tree.HasCharacter()) EncodingCharacters[current] =
90         tree.GetCharacter();
91 }
92 }

```

Listing 8: Code for the HuffmanDecoder class

References

- [1] Billy B. Brumley et al. “Practical realisation and elimination of an ECC-Related software bug attack”. In: *Proceedings of the 12th Conference on Topics in Cryptology*. CT-RSA’12. San Francisco, CA: Springer-Verlag, 2012, pp. 171–186. ISBN: 9783642279539. DOI: [10.1007/978-3-642-27954-6_11](https://doi.org/10.1007/978-3-642-27954-6_11). URL: https://doi.org/10.1007/978-3-642-27954-6_11.
- [2] kartik. *RSA Algorithm in Cryptography*. 23rd July 2025. URL: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/> (visited on 26/10/2025).
- [3] Andrew D. Ker. *Computer Security Lecture Notes*. <https://www.cs.ox.ac.uk/andrew.ker/docs/computersecurity-lecture-notes-mt2014.pdf>. 2014.
- [4] *Determine if a file has been modified*. Sept. 2019. URL: <https://unix.stackexchange.com/questions/176454/determine-if-a-file-has-been-modified> (visited on 30/10/2025).
- [5] Professor Vijay Raghunathan. *ECE264: Huffman Coding*. 2017. URL: <https://engineering.purdue.edu/ece264/17au/hw/HW13?alt=huffman> (visited on 10/11/2025).